

⚠ ERRORES TÍPICOS & REGLAS DE ORO

🔥 ERRORES MÁS FRECUENTES	
ERROR	CONSECUENCIA
<code>mappedBy</code> ausente	Tabla intermedia innecesaria
Lado dueño incorrecto	FK no se guarda en BD
<code>EAGER</code> en todo	N+1 queries, lentitud
<code>LAZY</code> sin sesión	<code>LazyInitializationEx</code>
Cascada mal puesta	Borrados accidentales
Sin <code>p.setUsuario(this)</code>	FK queda null en BD

- ✓ REGLAS DE ORO
- El **lado dueño** siempre tiene `@ManyToOne` o la FK `mappedBy` va en el lado *inverso* (`@OneToMany`)
 - Usa **LAZY** por defecto; EAGER solo si imprescindible
 - Sincroniza ambos lados con método `addX()`
 - `CascadeType.ALL` solo si el ciclo de vida es dependiente
 - Persiste desde el lado dueño o con `cascade = PERSIST`

- 🔍 LO QUE HACE JPA AUTOMÁTICAMENTE
- Crea las **claves foráneas**
 - Genera **tablas intermedias** en ManyToMany
 - Lanza **JOINS** al navegar relaciones
 - Sincroniza objetos relacionados con la BD
- ```
// pedido.getUsuario().getNombre() genera:
SELECT u.nombre FROM pedidos p
JOIN usuarios u ON p.usuario_id = u.id
```

💡 Idea final

Dominar las relaciones diferencia a quien *usa JPA* de quien lo *entiende*. Son el núcleo del ORM.

## ⚙ FETCHTYPE & CASCADETYPE

📖 FETCHTYPE – CUÁNDO SE CARGAN LOS DATOS

|                    | LAZY                   | EAGER             |
|--------------------|------------------------|-------------------|
| Carga              | Al acceder             | Al cargar entidad |
| Rendimiento        | ✓ eficiente            | ✗ lento si abusa  |
| Riesgo             | LazyInitEx             | N+1 queries       |
| Default @ManyToOne | EAGER → cambiar a LAZY |                   |
| Default @OneToMany | LAZY → mantener        |                   |

```
@ManyToOne(fetch = FetchType.LAZY)
private Usuario usuario;

@OneToMany(fetch = FetchType.EAGER)
private List<Pedido> pedidos;
```

🔄 CASCADETYPE – QUÉ SE PROPAGA

| TIPO    | PROPAGA EN...        |
|---------|----------------------|
| PERSIST | em.persist()         |
| REMOVE  | em.remove()          |
| MERGE   | em.merge()           |
| REFRESH | em.refresh()         |
| DETACH  | em.detach()          |
| ALL     | Todos los anteriores |

```
@OneToMany(
 mappedBy = "usuario",
 cascade = CascadeType.ALL
)
private List<Pedido> pedidos;
```

⚠ Con `REMOVE/ALL`: borrar Usuario borra todos sus Pedidos.

🏆 COMBINACIÓN RECOMENDADA

```
// Lado uno – dueño del ciclo de vida
@OneToMany(
 mappedBy = "usuario",
 cascade = CascadeType.ALL,
 fetch = FetchType.LAZY
)
private List<Pedido> pedidos;

// Lado muchos
@ManyToOne(fetch = FetchType.LAZY)
private Usuario usuario;
```

DAM · PROGRAMACIÓN · JPA / ORM



## Relaciones entre Entidades

@OneToMany · @ManyToOne · @OneToOne · @ManyToMany



## 🔗 TIPOS DE RELACIÓN & ONETOONE

### EL PROBLEMA DE LAS RELACIONES

En BD: **claves foráneas**. En Java: **referencias entre objetos**. JPA traduce entre ambos mundos.

Java Objects



BD Tablas FK

### 📄 LOS 4 TIPOS JPA

@OneToOne

@OneToMany

@ManyToOne

@ManyToMany

### 👤 ONETOONE — 1 A 1

Un **Usuario** tiene exactamente un **Perfil**.

usuarios: id, nombre | perfiles: id, bio, **usuario\_id FK**

```
@Entity
public class Usuario {
 @Id @GeneratedValue
 private int id;
 @OneToOne
 private Perfil perfil;
}
```

```
@Entity
public class Perfil {
 @Id @GeneratedValue
 private int id;
 private String bio;
 @OneToOne
 private Usuario usuario;
}
```

💡 Añade `@JoinColumn(name="usuario_id")` en el lado con la FK para evitar tabla intermedia.

### RESUMEN · ¿QUIÉN TIENE LA FK?

| ANOTACIÓN   | FK EN...         | MAPPEDBY EN... |
|-------------|------------------|----------------|
| @ManyToOne  | este lado ✓      | no lleva       |
| @OneToMany  | tabla del Many   | este lado ✓    |
| @OneToOne   | quien elijas     | el inverso ✓   |
| @ManyToMany | tabla intermedia | el inverso ✓   |

## ★ ONETOMANY / MANYTOONE - MAPPEDBY

La más importante del bloque. La FK está siempre en la tabla del lado *muchos*. El `@ManyToOne` es el dueño.

usuarios: id, nombre | pedidos: id, fecha, **usuario\_id FK**

### LADO MUCHOS — @MANYTOONE (DUEÑO)

```
@Entity
public class Pedido {
 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private int id;
 private String descripcion;

 @ManyToOne // lado dueño — FK aquí
 private Usuario usuario;
}
```

### LADO UNO — @ONETOMANY (INVERSO)

```
@Entity
public class Usuario {
 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private int id;
 private String nombre;

 @OneToMany(mappedBy = "usuario",
 cascade = CascadeType.ALL)
 private List<Pedido> pedidos = new ArrayList<>();

 // Helper — sincroniza ambos lados en memoria
 public void addPedido(Pedido p) {
 pedidos.add(p);
 p.setUsuario(this); // ← obligatorio
 }
}
```

### 🔑 MAPPEDBY — CONCEPTO CRÍTICO

`@OneToOne(mappedBy = "usuario")`

"La relación ya está gestionada por el atributo `usuario` en la entidad `Pedido`"

❌ Sin mappedBy

Tabla intermedia innecesaria

✅ Con mappedBy

Usa la FK real de la tabla

### ▶ MAIN — PERSISTIR

```
Usuario u = new Usuario();
u.setNombre("Ana");
Pedido p1 = new Pedido();
p1.setDescripcion("Pedido 1");
u.addPedido(p1); // sincroniza ambos lados
em.persist(u); // cascade guarda p1
```

## 🔗 MANYTOMANY & EJERCICIOS

### MANYTOMANY — N A N

Un **Pedido** tiene muchos **Productos** y viceversa.

pedidos: id, fecha · productos: id, nombre, precio  
**pedido\_producto:** pedido\_id FK, producto\_id FK ← **auto**

```
@Entity
public class Pedido {
 @Id @GeneratedValue
 private int id;

 @ManyToMany // lado dueño
 private List<Producto> productos;
}
```

```
@Entity
public class Producto {
 @Id @GeneratedValue
 private int id;
 private String nombre;
 private double precio;

 @ManyToMany(mappedBy = "productos")
 private List<Pedido> pedidos;
}
```

ORM crea automáticamente la tabla intermedia. Personalízala con `@JoinTable`.

### 📄 EJERCICIOS DEL BLOQUE

| #   | TAREA                                           | REL . |
|-----|-------------------------------------------------|-------|
| 1   | Usuario con lista de pedidos, persistir todo    | 1:N   |
| 2   | Pedido asociado a usuario existente             | N:1   |
| 3   | Agregar productos a pedidos                     | N:M   |
| 4   | Usuario + Pedido + Productos en una transacción | todo  |
| 5,8 | Usuario → Perfil, guardar ambos                 | 1:1   |
| 6   | addPedido en Usuario, guardar con 2 pedidos     | 1:N   |
| 7   | Solo Pedido con ManyToOne a usuario existente   | N:1   |
| 9   | Pedido → 2 Productos, persistir todo            | N:M   |

⚠️ Lado dueño: el que tiene la FK no lleva `mappedBy`, El inverso sí.